



Unidad 4

El modelo avisa cuando cambia

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026



El modelo tiene una lista de observadores y los llama después de cada cambio.

Los controladores escriben en el modelo. Las vistas leen del modelo, cuando el modelo avisa.

Dos ventanas, dos controladores

```
// ControladorCaja
private void registrar(Pago pago) {
    caja.registrar(pago);
    vista.mostrarTotal(caja.total());
    listado.actualizar();    // una linea por vista
}

// ControladorListado
botonAnular.addActionListener(e -> {
    caja.anularUltimo();
    listado.actualizar();    // y VistaCaja? No la conoce
});
```

Caja

Monto:

Total del día: \$2585.60

Pagos del día		
Medio	Monto	Acreditado
Efectivo	1500.00	1500.00
Debito	800.00	788.00
Anular último		

Cada controlador actualiza las vistas que conoce. Tres pagos, un anular: dos filas y un total de tres.



La Caja define su observador

```
// En el paquete modelo, igual que Notificador: lo que la Caja  
// necesita de una vista es un metodo para llamarla despues de cada cambio.  
public interface ObservadorCaja {  
    void cajaCambio(Caja caja);  
}  
  
public class Caja {  
    private final List<Pago> pagos;  
    private final List<ObservadorCaja> observadores;  
  
    public void agregarObservador(ObservadorCaja observador) {  
        this.observadores.add(observador);  
    }  
}
```

La Caja no importa `javax.swing` ni conoce ninguna vista: tiene una lista de `ObservadorCaja`.



La Caja avisa

```
public boolean registrar(Pago pago) {
    this.pagos.add(pago);
    this.notificar();           // al final de cada cambio
    return true;
}
public boolean anularUltimo() {
    this.pagos.remove(this.pagos.size() - 1);
    this.notificar();
    return true;
}
private void notificar() {
    for (ObservadorCaja o : this.observadores) o.cajaCambio(this);
}
```

Cada método que cambia el estado termina con `notificar()`. Quiénes son los observadores y qué hacen no es asunto de la Caja.

Las vistas observan

```
public class VistaCaja implements ObservadorCaja {  
    @Override  
    public void cajaCambio(Caja caja) {  
        mostrarTotal(caja.total());  
    }  
}  
  
// En Programa: las vistas se registran  
caja.agregarObservador(vista);  
caja.agregarObservador(listado);  
  
// En los controladores queda solo la llamada  
caja.registrar(pago);  
caja.anularUltimo();
```

Caja

Monto:

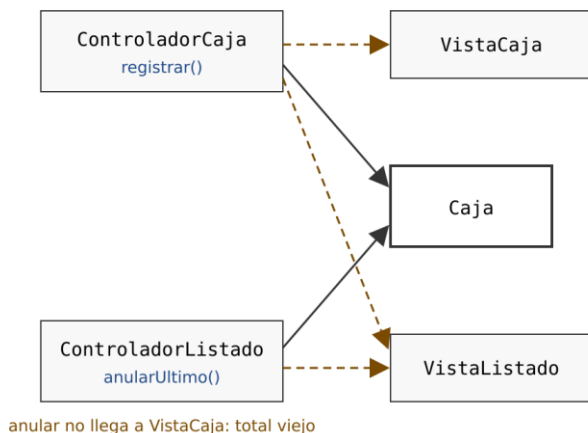
Total del día: \$2288.00

Pagos del día		
Medio	Monto	Acreditado
Efectivo	1500.00	1500.00
Debito	800.00	788.00
Anular último		

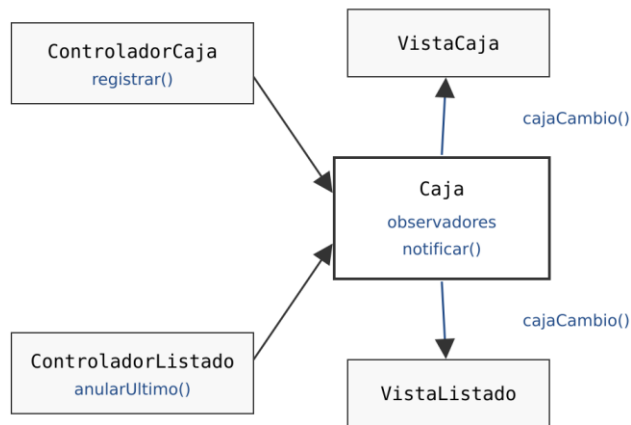
Los controladores escriben en la Caja. Las vistas leen de la Caja cuando la Caja avisa.

Antes y después

Antes: cada controlador actualiza sus vistas



Después: la Caja avisa a los observadores



Los controladores escriben en la Caja. Las vistas leen de la Caja, cuando la Caja avisa.

Una ventana nueva: una clase que implementa `ObservadorCaja` y una línea en Programa. Los controladores no cambian.

JTable y TableModel: ModeloTablaPagos extends AbstractTableModel

```
String[] COLUMNAS = {"Medio", "Monto", "Acreditado"};
Caja caja; // los datos siguen ahi
public int getRowCount() {
    return caja.getPagos().size();
}
public int getColumnCount() {return COLUMNAS.length;}
public String getColumnName(int c) {return COLUMNAS[c];}
public Object getValueAt(int fila, int col) {
    Pago pago = caja.getPagos().get(fila);
    if (col == 0) return pago.medio();
    if (col == 1) return pago.getMonto();
    return pago.montoAcreditado();
}
```

Pagos del dia		
Medio	Monto	Acreditado
Efectivo	1500.00	1500.00
Debito	800.00	788.00
QR	300.00	297.60
Anular ultimo		

Una `JTable` dibuja lo que responde su `TableModel`: filas, columnas, nombres y celdas. Los datos siguen en la `Caja`.



La tabla también observa

```
// En VistaListado
JTable tabla = new JTable(modeloTabla);
ventana.add(new JScrollPane(tabla), BorderLayout.CENTER); // encabezado

@Override
public void cajaCambio(Caja caja) {
    modeloTabla.fireTableDataChanged(); // avisa a la JTable
}

// La JTable es observadora de su TableModel (TableModelListener).
// fireTableDataChanged() es su notificar(): la tabla vuelve a pedir
// filas y celdas.
```

El mismo patrón, adentro de la biblioteca: `TableModel` avisa, `JTable` se redibuja.



Observer, con distintos nombres

- **JBUTTON**: `addActionListener()` registra, `actionPerformed()` es el aviso. El botón es el observado.
- **TableModel**: `addTableModelListener()`, `tableChanged()`. La **JTable** es la observadora.
- Modelo propio: **PropertyChangeSupport** con `addPropertyChangeListener()` y `firePropertyChange("total", viejo, nuevo)`. Misma lista, mismo `for`.
- Con dos o tres cambios distintos, una interfaz propia como **ObservadorCaja** queda más clara.

Observer: el observado tiene una lista de observadores y los llama cuando cambia.



Resumen

- Actualizar vistas desde el controlador: cada vista es una línea en cada método, y una línea que falta es una ventana vieja.
- El modelo define la interfaz del observador, guarda la lista y llama a **notificar()** al final de cada cambio.
- Las vistas implementan la interfaz y leen del modelo en el aviso. Escribir sigue siendo del controlador.
- **JTable** dibuja lo que responde su **TableModel**, y se redibuja con **fireTableDataChanged()**.
- **ActionListener**, **TableModelListener**, **PropertyChangeListener**: el mismo patrón.



La actividad de hoy

- El conversor, con un historial en el modelo y una segunda ventana con la tabla. El controlador actualiza las dos a mano, y en euros se olvida de una.
- **Parte 1:** convertir a dólares, a euros y a reales, y ver qué ventana queda vieja.
- **Parte 2:** el **Conversor** acepta observadores y avisa; las dos vistas implementan **ObservadorConversor**; el controlador solo pide conversiones.
- **Parte 3:** una tercera ventana, **VistaContador**, sin tocar el controlador.